

EC04 — Conception et développement d'une API sécurisée et documentée

Bloc 2 · Back-end · Coefficient 5 · /100 Durée : 4h00 · Épreuve individuelle, sur ordinateur

À lire avant de commencer — rassurez-vous

Cette épreuve évalue un ensemble large de compétences : il est **normal** de ne pas tout maîtriser parfaitement.

- **La non-atteinte d'un critère n'est pas éliminatoire.** Chaque critère est noté indépendamment.
- **La note de validation correspond à la moitié des points** (50/100). Vous pouvez très bien valider l'épreuve sans avoir tout terminé.
- **Mieux vaut faire moins, mais propre et compréhensible**, que tenter de tout couvrir avec un code qui ne tourne pas.
- Si vous bloquez sur un point, **expliquez votre raisonnement dans le rapport** : les choix justifiés rapportent des points même quand le code n'est pas finalisé.

Contexte

SkillHub est une plateforme de mise en relation entre formateurs indépendants et apprenants autour d'ateliers courts (cf. contexte général de la formation). On y trouve des **utilisateurs** (apprenants ou formateurs), des **ateliers** organisés par les formateurs, des **catégories** d'ateliers, et des **inscriptions** d'apprenants à ces ateliers.

La direction souhaite ouvrir une **API publique** consommable par le front-end React (cf. EC02), une future application mobile, et des partenaires tiers. Cette API doit être :

- **REST**, conforme aux bonnes pratiques (ressources, verbes, codes HTTP),
- **sécurisée** : authentification, autorisation, validation des entrées, gestion des erreurs,
- **documentée automatiquement** via **OpenAPI** (Swagger).

Votre mission : **concevoir l'architecture de l'API** et en **implémenter un sous-ensemble réduit mais propre** dans le temps imparti. L'épreuve est volontairement recentrée sur **3 piliers** : la conception REST, la sécurité, et la documentation OpenAPI.

Ressource fournie

Un fichier **skillhub.sql** (schéma + jeu de données minimal) vous est fourni au début de l'épreuve. Vous pouvez l'utiliser tel quel ou l'enrichir si votre conception le justifie. Vous n'êtes pas obligé-e de partir de cette base : si vous préférez (ré)utiliser votre propre modélisation (par exemple celle d'EC03), c'est possible — à condition de le mentionner et le justifier dans le rapport. Note: dans le fichier SQL fourni, tous les passwords sont "Password123!".

Travail à réaliser (3h sur machine)

1. Conception (45 min)

1.1 Cartographie des ressources

À partir du contexte SkillHub, **identifiez les ressources** que votre API va exposer. Listez-les avec leurs principaux attributs et les relations entre elles. Une cartographie courte mais cohérente suffit (pas besoin de tout modéliser).

1.2 Tableau des endpoints

Produisez un tableau listant **les endpoints que vous choisissez d'exposer** (3 à 4 au total), avec pour chacun :

- méthode HTTP,
- chemin,
- description,
- code de retour normal,
- codes d'erreur possibles,
- niveau d'autorisation requis (anonyme, utilisateur connecté, propriétaire, admin).

Périmètre attendu : 3 à 4 endpoints au total.

Endpoints imposés (chemin et verbe fixés) :

Méthode	Chemin	Description
POST	<code>/api/auth/register</code>	Création d'un compte
POST	<code>/api/auth/login</code>	Authentification, retour d'un token

À vous de choisir 1 à 2 endpoints métier supplémentaires, dont **au moins un endpoint non trivial** : c'est-à-dire un endpoint qui n'est **pas un simple CRUD à plat** sur une ressource. Par exemple, un endpoint qui agrège plusieurs ressources, applique un filtrage métier, ou expose une sous-collection liée à une ressource parente.

Exemple : `GET /api/workshops/{id}/participants` pour récupérer la liste des utilisateurs inscrits à un atelier donné. (C'est un exemple : vous êtes libre d'en proposer un autre.)

Important : mieux vaut **3 endpoints très propres** (REST cohérent, sécurité bien faite, doc OpenAPI claire) que 6 endpoints à moitié faits. Le scope est volontairement réduit.

2. Implémentation (2h15)

2.1 Stack technique — libre

Le **langage et le framework sont à votre choix**. Quelques exemples de stacks adaptées :

- **PHP** : Symfony + API Platform, Laravel, Slim.

- **JavaScript / TypeScript : NestJS** (équivalent le plus proche d'API Platform côté JS, avec décorateurs et génération OpenAPI via `@nestjs/swagger`), **AdonisJS**, **LoopBack 4**, **FoalTS**, ou Express / Fastify accompagnés de bibliothèques (`zod` ou `class-validator` pour la validation, `swagger-jsdoc` ou `tsoa` pour OpenAPI).
- **Python** : FastAPI, Django REST Framework, Flask.
- **Java / Kotlin** : Spring Boot (avec `springdoc-openapi`).
- autre choix raisonnable et défendable dans le rapport.

Très fortement conseillé : utilisez un **framework spécialisé API**. Tout faire « à la main » est possible mais peu réaliste sur la durée de l'épreuve. Choisissez un outil qui vous donne au minimum : routage, validation, sérialisation, et génération de doc OpenAPI.

Interdit : les **outils de génération automatique d'API à partir d'un schéma de base** (Backend-as-a-Service, BaaS), comme **Supabase**, **Firebase / Firestore**, **PostgREST**, **Hasura**, **Strapi**, **Directus**, **Pocketbase**, **Appwrite**, ou tout équivalent. Le but de l'épreuve est que vous **conceviez et écriviez** votre API ; ces outils la génèrent à votre place et vous priveraient de l'évaluation.

Côté base de données : utilisez le `skillhub.sql` fourni (PostgreSQL ou MariaDB/MySQL) ou votre propre BDD si vous préférez. **Docker / docker-compose autorisé** pour faire tourner la BDD si cela vous arrange : l'installation de l'environnement n'est pas évaluée.

2.2 Modèle de données

Modélisez (en code, via ORM ou via couche d'accès aux données de votre choix) au moins les ressources que vous avez décidé d'exposer dans votre API. Le mot de passe utilisateur **ne doit jamais être exposé** dans les réponses JSON.

2.3 Conception REST

- **Sérialisation** : séparation claire entre les champs lus et les champs écrits. Aucun champ sensible ne doit fuir (mot de passe, hash, secrets internes).
- **Validation** des entrées sur toutes les opérations d'écriture (formats, longueurs, types, email valide, etc.) avec retour d'erreurs structurées.
- **Codes HTTP** corrects (200, 201, 204, 400, 401, 403, 404, 422 selon les cas).

Bonus valorisés (non obligatoires) : pagination sur les collections, filtres (par recherche partielle ou champ exact) et tri sur au moins une ressource. À ne traiter que si les 3 piliers (conception REST, sécurité, OpenAPI) sont déjà solides.

2.4 Sécurité

- **Authentification** par token (JWT recommandé, mais autre schéma à token autorisé si défendu dans le rapport) :
 - `POST /api/auth/register` : création de compte, ouvert aux anonymes, **mot de passe haché** côté serveur (l'algorithme de hash est au choix tant qu'il est adapté — pas de SHA1/MD5 nus).
 - `POST /api/auth/login` : retour d'un token (durée raisonnable, par exemple 1h).
- **Autorisations fines** : en plus de la simple distinction « connecté / pas connecté », au moins **deux niveaux d'autorisation métier** différents doivent être implémentés (par exemple : seul le

propriétaire d'un atelier peut le modifier ou le supprimer ; seul l'utilisateur concerné peut annuler sa propre inscription ; etc.).

- **Politique de mot de passe** minimale à l'inscription (par ex. 8 caractères, 1 majuscule, 1 chiffre).

2.5 Documentation

- **Documentation OpenAPI / Swagger** générée et accessible (par ex. `/api/docs` ou route équivalente selon votre stack).
- Pour chaque endpoint que vous exposez, fournir une **description et un exemple** lisibles dans la doc.
- Export du fichier `openapi.json` ou `openapi.yaml` dans le rendu (`docs/openapi.{json|yaml}`).

Si votre stack ne génère pas la doc automatiquement, vous pouvez la rédiger à la main — mais c'est long. C'est une raison de plus de choisir un framework qui le fait pour vous.

2.6 Tests automatisés (bonus valorisé)

La présence de tests automatisés est **fortement valorisée** mais n'est pas obligatoire. Si vous en écrivez, couvrez en priorité :

1. l'inscription d'un utilisateur,
2. le login et la récupération d'un token,
3. un appel authentifié réussi,
4. un appel refusé pour cause d'autorisation manquante (403).

Joindre la sortie des tests dans le rapport.

2.7 Collection Postman / Bruno (bonus)

Fournir une collection Postman ou Bruno avec :

- requête de login,
- variable d'environnement `{{token}}` automatiquement renseignée,
- requêtes pour chaque endpoint exposé.

Livrables (à remettre dans `EC04_NomPrenom.zip`)

```
EC04_NomPrenom.zip
├── conception/
│   └── endpoints.md
├── src/                                # code source du projet (sans dépendances)
├── docs/
│   └── openapi.{json|yaml}
├── tests/                             # tests automatisés (si présents)
├── postman/                           # collection Postman / Bruno (si présente)
│   └── skillhub-api.postman_collection.json
```

```
├── .env.dist
└── README.md
```

⚠ **NE PAS inclure** les dossiers de dépendances (`vendor/`, `node_modules/`, `.venv/`, etc.) ni les caches.

Rapport (1h)

Document `README.md` (ou PDF) **court : 2 à 3 pages**, structuré en **4 sections** :

1. **Stack technique** : langage, framework API, lib d'authentification, et justification rapide du choix.
2. **Sécurité** : flux d'authentification (en quelques lignes ou un schéma texte simple), durée du token, et description des **2 niveaux d'autorisation métier** mis en place. Présentation et justification de l'**endpoint non trivial** retenu.
3. **Documentation OpenAPI** : capture annotée de la doc générée, et instructions d'accès (URL locale, commandes pour lancer le serveur, préparation BDD à partir de `skillhub.sql`).
4. **Limites et améliorations** : ce que vous n'avez pas eu le temps de faire, et ce qui irait dans une vraie API de production (refresh token, rate limiting, pagination, observabilité, etc.).

Note pour les correcteurs : la priorité de l'évaluation porte sur la lecture du code, du tableau d'endpoints et de la doc OpenAPI. L'installation et l'exécution effective du serveur restent possibles si un doute subsiste, mais ne sont pas systématiques.

Critères d'évaluation (barème /100)

Critère	Points
Cartographie des ressources et tableau des endpoints (cohérence, complétude raisonnable)	10
Endpoint non trivial (pertinence, qualité de l'implémentation)	10
Modèle de données et exposition (séparation lecture/écriture, aucun champ sensible exposé)	10
Validation des payloads et codes HTTP cohérents	10
Authentification (register, login, hash du mot de passe, token)	15
Autorisations fines (au moins 2 niveaux métier)	15
Documentation OpenAPI exploitable	15
Rapport (analyse, justifications, qualité globale)	10
Qualité du code (lisibilité, séparation des responsabilités)	5
Bonus : pagination / filtres / tri, tests automatisés, collection Postman/Bruno, refresh token, rate limiting, etc.	jusqu'à +10

Validation : $\geq 50 / 100$ **Rattrapage** : entre 35 et 50 **Non validé** : < 35

Contraintes et interdictions

- **Libre** : langage et framework au choix. L'utilisation d'un framework API est très fortement conseillée.
 - **Imposé** : authentification par token (JWT recommandé), documentation OpenAPI/Swagger, BDD relationnelle (PostgreSQL ou MariaDB/MySQL).
 - **Interdit** : outils de génération automatique d'API à partir d'un schéma de BDD (Supabase, Firebase, PostgREST, Hasura, Strapi, Directus, Pocketbase, Appwrite, etc.).
 - **Interdit** : exposer `password`, `salt` ou tout champ sensible dans les réponses JSON.
 - **Interdit** : désactiver les contrôles d'autorisation par convenance (ex. mode debug avec sécurité bypassée).
 - **Internet autorisé** pour la documentation officielle de votre stack. Interdit : ChatGPT, Copilot, Claude et tout outil de génération de code.
-

Conseils

- **Restez dans le scope** : 3 à 4 endpoints au total, c'est largement suffisant pour démontrer ce qui est évalué. N'essayez pas de couvrir toute la plateforme.
- **Lisez la doc de votre framework API avant de coder** : les bons frameworks vous donnent gratuitement la moitié de l'épreuve (sérialisation, validation, OpenAPI).
- **Configurez l'authentification tôt** : sans token fonctionnel, vous ne pouvez pas tester les autorisations.
- **Séparez clairement les champs lus et les champs écrits** dès le début : c'est la clé pour ne jamais leaker le mot de passe.
- **Testez vos endpoints au fur et à mesure** (Postman, Bruno, ou simple `curl`) : ne découvrez pas les bugs à la fin.
- **Si le temps presse**, sacrifiez les bonus (pagination, filtres, tri, tests automatisés, collection Postman), **pas la sécurité ni la documentation**.